# COMP2522

Lesson 6: Functional Interfaces, Lambda Expressions

# Learning Outcomes

- Functional interfaces
- Lambda expressions

# Functional Interface

- **A functional interface is an interface that contains just a single abstract method** (note: it may also contain default and static methods, but only ONE abstract).
- We may declare this interface with the @FunctionalInterface annotation if we like (which will complain if we don't have exactly ONE abstract method).

- Because there is only one abstract method, when that interface gets implemented, it is obvious which method must be created. Java will know the types and number and order of parameters, and the datatype that method returns also (if any). Because of this, we can skip the @Override full implementation of that method and instead use a shortcut: a lambda expression.

- Therefore, a lambda expression is the implementation of a functional interface's single abstract method, in a way that makes it readable, flexible, and short.

# Lambda Expression: Introduction

- Functional interfaces can be implemented by lambda expressions by creating an anonymous function. This function exists without actually belonging to any class.
- Reasons to use lambda expressions:

1. This function can even be passed as a parameter (like an object) to methods and executed on demand.
2. We can write functional programming instead of / in addition to OOP
3. Our code will be short yet very readable too.

- The syntax for writing a lambda expression is: (arg list)->{code}. For example:
- ()->single-line-of code
- ()->{multiple "lines of code"}
- (a1)->{code using a1, whose type is known by the Functional Interface's method signature}
- (Type a2)->{code using a2, with the type explicitly declared}
- (a3, a4, a5, a6)->{code using a3, a4, a5, a6}

# Example

Examples; note how at runtime we re-define the abstract method over and over again according to our needs:

```
@FunctionalInterface
interface Nameable{
    String getOneString(String s1, String s2, int n);
}

class Main{
    public static void main(final String[] args)    {
        // repeat strings n times
        Nameable repeatNames = (first, last, repeat)->{String s =""; for(int i = 0; i < repeat; i++){s+=first;s+=last;} return s;};
        System.out.println(repeatNames.getOneString("tiger", "woods", 3));

        // string gets first n chars
        Nameable getSubstrings = (string1, string2, n)->{String s=""; s+=string1.substring(0, n); s+=string2.substring(0, n);return
s;};
        System.out.println(getSubstrings.getOneString("tiger", "woods", 3));

        // get letters at position n
        Nameable nthChars = (str1, str2, n)->{return "" + str1.charAt(n) + str2.charAt(n);};
        System.out.println(nthChars.getOneString("tiger", "woods", 3));
    }
}
```

# Syntax

- Syntax rules for lambda expressions:
- ()->{}  // (parentheses) needed if no parameters
- x->{}  // brackets not needed if there is a single parameter, unless:
- (int x)->{}  // brackets needed if you choose to define the datatype for a single parameter
- (x, y)->{}  // brackets needed for multiple parameters
- x->code  // curly braces not needed if just one single instruction and the evaluation of the code matches the method's return type

# Lamba Examples

```
@FunctionalInterface
interface Mathable
{
   double doMath(int op1, int op2);
}

class MathStuff
{
   public static void main(final String[] args)
   {
      Mathable add = (a,b)->a+b;
      System.out.println(add.doMath(2, 6));                    // 8.0

      Mathable power = (x,y)->Math.pow(x,y);
// or power = (x,y)->{return Math.pow(x,y);}
      System.out.println(power.doMath(2, 6));                  // 64.0
   }
}
```

# Java’s Functional Interfaces

The java.lang.Iterable interface is a Functional Interface; but we are interested in its default forEach method, which takes one parameter: a Consumer Functional Interface which has one abstract method: void accept(T t); in other words, the forEach method can take a lambda expression that takes any single type as an

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
class BookStore{
   public static void main(final String[] args)    {
      List<String> titles;
      titles = new ArrayList<>();
      String[] data = {"Anna Karenina", "Madame Bovary", "War and Peace",
"The Great Gatsby",
                           "Lolita", "Middlemarch",  "The Adventures of Huckleberry
Finn",
      "The Stories of Anton Chekhov", "In Search of Lost Time", "Hamlet"};
      titles = Arrays.stream(data).toList();
      titles.forEach(title -> {
         if(title.length() < 10)
         {
            System.out.println(title);
         }
      });
      titles.forEach(t -> {
         if(t.contains("of"))
         {
            System.out.println(t);
         }
      });
   }
}
```

# Lambda (and a bit of method reference!)

```
import java.util.ArrayList;
import java.util.List;
public class Notebook{
    public static void main(final String[] args)    {
        final List<String> notes;
        notes = new ArrayList<>();
        notes.add("Work Hard");
        notes.add("Have Fun");
        //display(notes, Notebook::upperIt);
        display(notes, s -> s.toUpperCase());
    }
    private static String lowerIt(String input)    {
        return input.toLowerCase();
    }
    private static String upperIt(String input)    {
        return input.toUpperCase();
    }
    private static void display(List<String> list, Converter converter)    {
        for(String s: list)        {
            System.out.println(converter.convert(s));
        }
    }
}
interface Converter{
    String convert(String input);
}
```

# Lambda Expression Examples

```
@FunctionalInterface
interface Mathable
{
   double doMath(int op1, int op2);
}

class MathStuff
{
   public static void main(final String[] args)
   {
      Mathable add = (a,b)->a+b;
      System.out.println(add.doMath(1, 40));

      Mathable power = (x,y)->Math.pow(x,y);
      System.out.println(power.doMath(2, 6));
   }
}
```

# More Examples

```
@FunctionalInterface
interface Importantable{
   String getImportantValue();
}

interface Mathyable{
   double doOperation(double x, double y);
}
class Arithmetic{
   double add(Mathyable m, double a, double b)    {
      return m.doOperation(a, b);
   }

   double subtract(Mathyable m, double a, double b)    {
      return m.doOperation(a, b);
   }
   double getRectangleArea(Mathyable m, double a, double b)    {
      return m.doOperation(a, b);
   }
}
```

```
class Whatever {

   public static void main(final String[] args ) {

      Importantable pi;
      pi = () -> "3.14159";
      System.out.println("Value of Pi = " + pi.getImportantValue());

      Importantable bcSchools;
      bcSchools = ()->"bcit";
      System.out.println("important BC school = " +
bcSchools.getImportantValue());

      String first = "tiger";
      String last = "woods";
      Importantable fullName;
      fullName = ()->first+" " + last;
      System.out.println("important name = " +
fullName.getImportantValue());

      Arithmetic a = new Arithmetic();
      System.out.println(a.add((c,d)->c+d, 7, 8));
      System.out.println(a.subtract((c,d)->c-d, 7, 8));
      System.out.println(a.getRectangleArea((c,d)->c*d, 7, 8));
   }
}
```

# Redefining at Runtime

```
@FunctionalInterface
interface Nameable
{
   String getOneString(String s1, String s2, int n);
}

class Main
{
   public static void main(final String[] args)
   {
      // repeat strings n times
      Nameable repeatNames = (first, last, repeat)->{String s =""; for(int i = 0; i < repeat; i++){s+=first;s+=last;} return s;};
      System.out.println(repeatNames.getOneString("tiger", "woods", 3));

      // string gets first n chars
      Nameable getSubstrings = (string1, string2, numLetters)->{String s=""; s+=string1.substring(0, numLetters);
s+=string2.substring(0, numLetters);return s;};
      System.out.println(getSubstrings.getOneString("tiger", "woods", 3));

      // get letters at position n
      Nameable nthChars = (str1, str2, n)->{return "" + str1.charAt(n) + str2.charAt(n);};
      System.out.println(nthChars.getOneString("tiger", "woods", 3));
   }
}
```

# Final Example

```
import org.w3c.dom.ls.LSOutput;

interface Caseable{
   String changeCase(String s);
}
class StringStuff{
   public static void main(String[] args)
   {
      Caseable upper = s->s.toUpperCase();
      Caseable lower = s->s.toLowerCase();
      Caseable title = s ->{
         return s.toUpperCase().charAt(0) +
                s.toLowerCase().substring(1);
      };

      Person p = new Person("tigEr");
      System.out.println(p.getName(upper));
      System.out.println(p.getName(lower));
      System.out.println(p.getName(title));
   }
}
class Person{
   private String name;
   Person(String n){
      name = n;
   }
   String getName(Caseable c){
      return c.changeCase(name);
   }
}
```

# Use Java’s “built-in” Functional Interfaces

- https://docs.oracle.com/en/java/javase/22/docs/api/java.base/java/util/function/package-summary.html

# Lambda Expressions

- We can use lambda expressions to shorten our code substantially.
- When you see the → arrow operator, you are likely seeing a lambda expression.

```
class Person {
    private final String name;
    private final int yearBorn;
    public Person(final String name, final int yearBorn) {
        this.name = name;
        this.yearBorn = yearBorn;
    }
    @Override
    public String toString() {
        return name + " (" + yearBorn + ")";
    }

    public static void main(final String[] args) {
        final List<Person> people;
        people = new ArrayList<>();
        people.add(new Person("Alice", 2000));
        people.add(new Person("Bob", 1999));
        people.add(new Person("Charlie", 1988));

        // Sort by year of birth
        Collections.sort(people, (p1, p2) -> p1.yearBorn -
p2.yearBorn);
        System.out.println("Sorted by age: " + people);

        // Sort by name using another Comparator
        Collections.sort(people, (p1, p2) ->
p1.name.compareTo(p2.name));
        System.out.println("Sorted by name: " + people);
    }
}
```

# Lambda Expressions

- In modern Java (since Java 8), it's often better and more concise to **use lambda expressions instead of anonymous inner classes**, especially for functional interfaces (interfaces with a single abstract method) like Comparator.
- Lambdas provide a more readable and concise way to implement functional interfaces.
- Anonymous inner classes are still necessary in cases where you're implementing interfaces with multiple methods or using non-functional interfaces.

```
public class Main {
    public static void main(final String[] args) {
        // Create a list of integers
        List<Integer> numbers = new ArrayList<>();
        numbers.add(10);
        numbers.add(5);
        numbers.add(20);
        numbers.add(15);

        // Sort the list in descending order using a lambda
expression
        Collections.sort(numbers, (a, b) -> b - a);

        // Print the sorted list
        System.out.println(numbers);  // Output: [20, 15, 10, 5]
    }
}
```